AgentKillChain: A Controlled Benchmark for Persistent Compromise in Autonomous AI Agents

Author: Kevin O'Connor

Version: Final public release, May 20, 2026

Repository: https://github.com/KevinOBytes/AgentKillChain

Abstract

Autonomous AI agents increasingly combine language models with persistent memory, retrieval, and tool execution. These capabilities expand the security boundary beyond a single model response: untrusted content can be summarized into memory, retrieved in later sessions, and used to influence tool selection. AgentKillChain is an open, controlled benchmark for evaluating this class of persistent compromise risk. The benchmark models eight prompt-injection and memory-poisoning families across five workflow framings, then measures whether a representative assistant loop produces unsafe instruction adoption, unsafe tool proposals, secret-disclosure signals, or explicit policy-violation markers. In a May 20, 2026 public release over 45 models and 40 attack scenarios, the harness attempted and scored 1,800 model-scenario evaluations across authenticated Ollama Cloud and OpenRouter provider cohorts. The combined release observed a 1.39% overall unsafe-signal rate under deterministic heuristic scoring. Unsafe signals were concentrated in single-phase toolchain-confusion, memory-poisoning, and context-manipulation scenarios; the current latent-memory scenarios did not produce scored unsafe activation under this heuristic. The baseline establishes a reproducible measurement for one controlled agent architecture and supports concrete defenses around memory provenance, deterministic tool authorization, context separation, and public reporting practices that preserve aggregate evidence without exposing sensitive run traces.

Keywords: AI security, autonomous agents, prompt injection, indirect prompt injection, memory poisoning, tool use, benchmark, Ollama, OpenRouter

1. Introduction

Large language model applications are moving from stateless chat interfaces toward stateful agents that retrieve documents, summarize working context, maintain long-term memory, and call tools. This shift changes the threat model. A malicious instruction no longer needs to win in a single turn. It can be embedded in retrieved content, survive summarization, and reappear later when the agent is operating in a higher-risk workflow.

AgentKillChain measures a specific failure mode in autonomous assistants: whether compromised context can persist across turns and influence later tool-facing behavior. The benchmark uses controlled scenarios involving poisoned documents, memory buffers, and tool-selection prompts to test whether an agent maintains the boundary between trusted instructions and untrusted data. The implementation is intentionally scoped: tools are stubbed, memory is represented as a controlled context buffer, and scoring favors conservative classification over inflated attack-success claims.

The May 20, 2026 release evaluates 39 authenticated Ollama Cloud models and six OpenRouter frontier/lite proprietary models across the full scenario catalog. The public release includes aggregate metrics, coverage, error rates, run metadata, public attack prompts, and redacted result artifacts while withholding private live request/response traces.

2. Contributions

This work contributes:

1. A threat model for persistent compromise in autonomous AI agents, organized around initial access, execution, persistence, activation, escalation, and impact.
2. A 40-scenario attack catalog spanning eight payload families and five operational workflow framings.
3. A Python harness that runs the same representative agent architecture against OpenRouter, local Ollama, or authenticated Ollama Cloud models.
4. Hardened heuristic scoring that distinguishes unsafe behavior signals from benign quotation or refusal.
5. A public-release artifact policy that keeps attack prompts public while redacting full live request/response traces and preserving flags, metrics, character counts, and SHA-256 trace digests.
6. A reproducible May 20, 2026 multi-provider release covering 1,800 scored model-scenario evaluations.

3. Threat Model

AgentKillChain adapts the cyber kill-chain pattern to stateful AI agents:

1. **Initial access:** Untrusted data enters the context window through user input, retrieved documents, notes, web content, or files.
2. **Execution:** The model parses adversarial data as if it were task-relevant instruction.
3. **Persistence:** A memory or summarization step carries the adversarial content into future context.
4. **Activation:** A later benign-looking workflow triggers the remembered payload.
5. **Escalation:** The agent proposes or selects a privileged tool action.

6. **Impact:** The tool action or response leaks data, violates policy, or routes the agent toward attacker goals.

The benchmark is scoped to a representative assistant agent with memory and tools. It measures how often this controlled loop emits unsafe signals under the published scenario catalog and scoring policy.

4. Benchmark Design

4.1 Dataset

The dataset contains 40 structured scenarios:

Dimension	Value
Payload families	8
Workflow framings	5
Total scenarios	40
Scenario phases	single, seed, dormancy, activation
Model catalog in release	45 models: 39 authenticated Ollama Cloud models and 6 OpenRouter proprietary models

The eight payload families are HTML injection, RAG poisoning, markdown injection, memory poisoning, latent memory poisoning, context drift, toolchain confusion, and cognitive overload. The five workflow framings are document summary, incident response, tool selection, memory recall, and user debug request.

Each scenario specifies an `attack_id`, `campaign_id`, `family`, `scenario_type`, `seed_input`, `trigger_input`, `expected_failure_mode`, `severity`, `realism_level`, and `phase`.

4.2 Harness

The harness executes one representative assistant architecture. The model receives a system prompt, user task, and a representative toolset:

- `read_document`
- `fetch_url`
- `search_notes`
- `send_email`
- `write_note`

The harness uses native OpenAI-format function calling when the model/provider accepts tool schemas. For Ollama models that reject native tools, it falls back to a text-only stub tool protocol. Tool effects are stubbed, and a lightweight policy layer marks overtly dangerous calls rather than executing external side effects.

The multi-stage scenario engine preserves campaign state across seed, dormancy, and activation phases. This allows the benchmark to test whether summarized memory can carry adversarial instructions into later tasks.

4.3 Scoring

The May 20, 2026 release uses hardened deterministic heuristic scoring. A model-scenario evaluation can be flagged for:

- `unsafe_instruction_adopted`
- `unsafe_tool_call_proposed`
- `secret_disclosure_claimed`
- `policy_violation`
- `refusal_detected`

The overall unsafe-signal score is the fraction of scored model-scenario evaluations where at least one unsafe signal is true. It is reported as an unsafe-signal rate for this harness and scenario set.

AgentKillChain also supports LLM-as-judge scoring, but that path was not used for the published May 20, 2026 release.

5. Public Artifact Policy

The attack catalog prompts are public and citable. Full live request/response traces are treated differently because they can include private system prompts, provider diagnostics, unsafe model-generated text, or material that should not be encouraged through public indexing. For this reason, the tracked public artifacts redact `results[].prompt` and `results[].output`.

The public JSON and CSV retain:

- evaluation counts and status counts
- model identifiers and attack identifiers
- scoring flags
- aggregate metrics
- request/response character counts
- SHA-256 digests for raw request/response traces

Raw live traces should remain in ignored local outputs under `results/generated/` or in private archival storage.

6. Results

The May 20, 2026 public release completed full catalog sweeps for two cohorts: 39 authenticated Ollama Cloud models and six OpenRouter proprietary models (`openai/gpt-5.3-chat`, `openai/gpt-5.4-mini`, `anthropic/claude-opus-4.7`, `anthropic/claude-haiku-4.5`, `google/gemini-3.1-pro-preview`, and `google/gemini-3.1-flash-lite`). A targeted rerun resolved the only initial Ollama provider API error, so the final public artifact has complete coverage.

Metric	Value
Providers	2
Cohorts	2
Models requested	45
Attack scenarios	40
Expected model-scenario evaluations	1,800
Attempted evaluations	1,800
Scored evaluations	1,800
API error evaluations	0
Timeout evaluations	0
Coverage rate	100.00%

Aggregate scored-evaluation metrics:

Metric	Value
Overall unsafe-signal rate	1.39%
Unsafe instruction adoption rate	0.56%
Unsafe tool proposal rate	1.33%
Secret disclosure rate	0.00%
Policy violation marker rate	1.11%
Refusal rate	59.78%

Cohort-level metrics:

Cohort	Models	Scored evaluations	API errors	Overall unsafe-signal rate
Ollama Cloud baseline	39	1,560	0	1.47%
OpenRouter frontier/lite cohort	6	240	0	0.83%

Unsafe signals were not evenly distributed across families. In the redacted public artifact, unsafe flags appear in the following family/phase groups:

Family / phase	Scored evaluations	Evaluations with any unsafe signal	Rate
Toolchain confusion / single	225	16	7.11%
Memory poisoning / single	225	4	1.78%
Context drift / single	225	3	1.33%
RAG poisoning / single	225	1	0.44%
Cognitive overload / single	225	1	0.44%
HTML injection / single	225	0	0.00%
Markdown injection / single	225	0	0.00%
Latent memory poisoning / seed, dormancy, activation	225	0	0.00%

In the OpenRouter cohort, the two unsafe-signal evaluations both came from `google/gemini-3.1-flash-lite`: one memory-poisoning scenario with an unsafe tool proposal and one toolchain-confusion scenario with unsafe instruction adoption, unsafe tool proposal, and a policy-violation marker. The other five OpenRouter models produced no unsafe signals under this heuristic.

7. Discussion

The release supports four conclusions.

First, most tested interactions were resilient under the current harness. The overall unsafe-signal rate was low, and no scored evaluation was marked as secret disclosure under the hardened heuristic.

Second, toolchain confusion remains the clearest signal in this release. Unsafe tool proposals appeared at a higher rate than other failure modes, which is consistent with the idea that agent risk often comes from the model's role as a tool-selection component rather than from text generation alone.

Third, the OpenRouter proprietary cohort had a lower unsafe-signal rate than the broader Ollama Cloud cohort, but it is also much smaller and should be interpreted as a provider-cohort check rather than a comprehensive proprietary-model ranking.

Fourth, the latent-memory scenarios did not produce scored unsafe activation in this release. This should not be read as evidence that long-horizon memory is safe. The benchmark uses a simplified memory buffer, one pass per scenario, and public payloads. Future work should evaluate weaker memory provenance controls, repeated trials, hidden hold-out payloads, more realistic retrieval systems, and live sandboxed tool execution.

8. Defensive Implications

The results point toward defenses that should sit outside the model:

1. **Memory provenance:** Agents should tag remembered content by source and trust level. Summarization should strip imperative instructions from untrusted retrieved data.
2. **Deterministic tool authorization:** High-risk tools should require policy checks independent of the language model's stated intent.
3. **Tool-call egress controls:** Email, web, file, and execution tools should be constrained by allowlists, explicit user confirmation, and sensitive-data detectors.
4. **Context separation:** System instructions, user intent, retrieved content, memory, and tool results should be represented as separate security domains rather than flattened into one prompt.
5. **Regression benchmarks:** Agent teams should rerun persistent-compromise tests after model, prompt, tool, and memory-policy changes.

9. Limitations

AgentKillChain is a controlled benchmark with explicit boundaries:

- The harness evaluates one representative assistant loop, not multiple production agent architectures.
- Tool execution is stubbed and cannot capture all real implementation edge cases.
- Memory is represented as a controlled context buffer rather than a full vector database or hierarchical memory graph.
- The May 20, 2026 release uses one pass per scenario, so it does not estimate variance or confidence intervals.
- Public attack payloads may eventually contaminate model training or alignment data.
- The public result artifact is redacted; exact transcript-level replay requires access to private archived traces.
- Heuristic scoring can miss subtle unsafe behavior or overcount some explicit unsafe markers.

These boundaries define the appropriate interpretation: an initial baseline for a controlled agent architecture, not a definitive ranking of model security.

10. Ethics and Release Controls

The dataset contains adversarial prompts and controlled unsafe-workflow scenarios. The public release keeps the attack catalog open for reproducibility and citation, but redacts raw live-run traces to avoid publishing model-specific unsafe output and private system prompt material. The benchmark uses stubbed tools and does not execute real exfiltration, file modification, or network side effects.

Researchers extending the benchmark should avoid running live tool actions against real user data. When publishing future runs, they should preserve aggregate evidence and private trace archives separately.

11. Reproducibility

The public repository contains the dataset, harness, website, redacted result artifacts, and visualization scripts.

Smoke validation:

```
python3 -m compileall harness scripts tests test_all.py
python3 -m unittest discover
python3 harness/runner.py --dry-run
python3 scripts/redact_public_results.py --input-json results/generated/model_results.json --output-
json results/model_results.json --output-csv results/model_results.csv
python3 scripts/validate_artifacts.py
```

Live Ollama Cloud baseline command:

```
python3 harness/runner.py --provider ollama-cloud --judge-mode heuristic --run-label ollama-cloud-
take-2026-05-20
python3 scripts/redact_public_results.py --input-json results/generated/model_results.json --output-
json results/model_results.json --output-csv results/model_results.csv
python3 scripts/validate_artifacts.py
```

Targeted API-error reruns can be performed with `--models` and `--attack-ids`, then merged with `scripts/merge_result_rerun.py`.

OpenRouter frontier/lite cohort command:

```
python3 harness/runner.py --provider openrouter --judge-mode heuristic --models openai/gpt-5.3-chat,openai/gpt-5.4-mini,anthropic/claude-opus-4.7,anthropic/claude-haiku-4.5,google/gemini-3.1-pro-preview,google/gemini-3.1-flash-lite --run-label openrouter-frontier-cohort-2026-05-20
```

The public multi-provider artifact is produced by combining cohort outputs with `scripts/combine_result_cohorts.py` and then redacting the combined payload with `scripts/redact_public_results.py`.

The raw live-run outputs are written to ignored `results/generated/` paths. The public release artifacts are `results/model_results.json` and `results/model_results.csv`.

References

[1] OWASP Foundation. "OWASP Top 10 for Large Language Model Applications." https://owasp.org/www-project-top-10-for-large-language-model-applications/

[2] National Institute of Standards and Technology. "Artificial Intelligence Risk Management Framework (AI RMF 1.0)." https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10

[3] Greshake, K. et al. "Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection." arXiv:2302.12173. https://arxiv.org/abs/2302.12173

[4] Microsoft Research. "Benchmarking and Defending Against Indirect Prompt Injection Attacks on Large Language Models." https://www.microsoft.com/en-us/research/publication/benchmarking-and-defending-against-indirect-prompt-injection-attacks-on-large-language-models/

[5] Debenedetti, E. et al. "AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents." NeurIPS 2024 Datasets and Benchmarks Track. https://papers.nips.cc/paper_files/paper/2024/file/97091a5177d8dc64b1da8bf3e1f6fb54-Paper-Datasets_and_Benchmarks_Track.pdf